MLOps with MLflow

A Living Hands-On Manual

From Environment Isolation to Reproducible ML Systems

Designed as an extensible teaching and laboratory companion. Each new practical can be added as an independent chapter without disturbing the conceptual spine of the manual.

Course Edition: 2026

How to Use This Living Manual

This manual is intentionally built as a *living course artifact*, not as a one-time set of notes. The conceptual narrative explains not only *what* to type, but also *what the operating system and Python are doing underneath*, why the step matters in MLOps, how to verify it, and what misconception students commonly develop.

Each chapter follows a reusable pattern: **Concept** → **Mechanism** → **Command** → **Visual Model** → **Verification** → **MLOps Connection** → **Classroom Checkpoint**. New chapters can therefore extend the course from local environments to MLflow tracking, model packaging, registries, deployment, CI/CD, observability, and governance.

Contents

Chapter 1

Environment Foundations for MLOps

Learning Lens

By the end of this chapter, a student should be able to distinguish a project directory from a Python virtual environment, explain why dependency isolation matters, create a virtual environment, interpret its directory structure, and connect environment control to reproducibility in MLOps.

1.1 The Problem Before the Tool

A machine may contain one global Python installation and many independent ML projects. If every project installs packages into the same global environment, a package upgrade required by one project can silently alter another. This is a dependency-management problem before it becomes an MLflow problem.

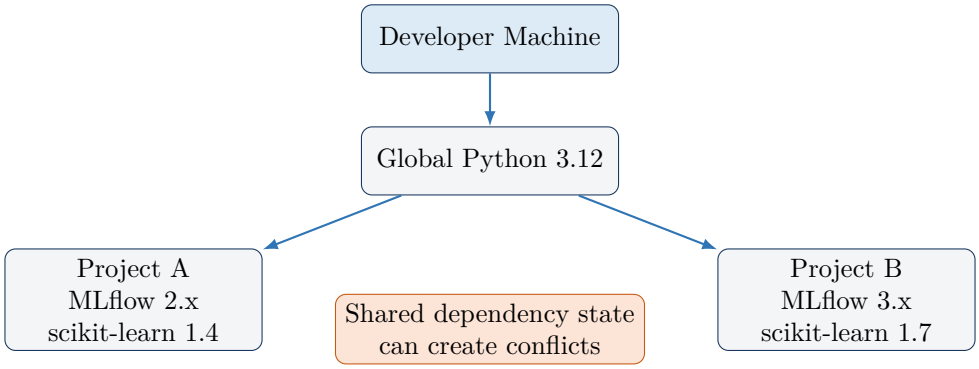


Figure 1.1: Why a single shared Python environment becomes fragile as projects diverge.

1.2 Virtual Environment as a Project Boundary

A **virtual environment** is a project-specific Python execution environment represented by a directory. It provides project-local interpreter entry points, package installation locations, and scripts. The central idea is isolation: Project A should be able to evolve without unintentionally changing Project B.

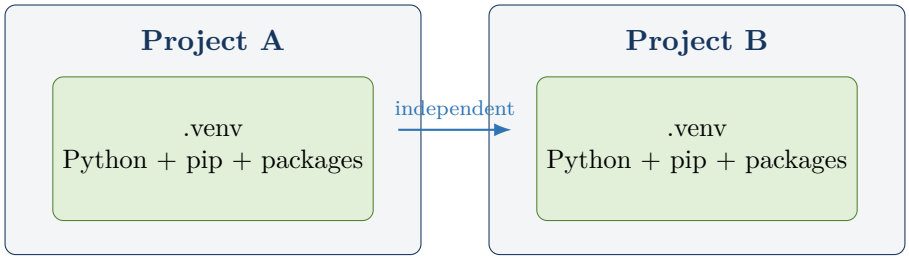


Figure 1.2: Isolation turns dependencies into project-local concerns.

MLOps Connection

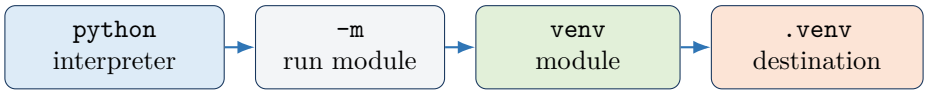
MLOps is concerned with repeatable behavior across development, experimentation, testing, and production. A virtual environment does not solve all reproducibility problems, but it establishes an essential boundary: the software dependencies of one project can be controlled independently of another.

1.3 Creating the Environment

The standard command is:

```
python -m venv .venv
```

Read the command from left to right:



Thus, the command means: *use this Python interpreter to execute Python's* `venv` *module and create the resulting environment in a directory named* `.venv`.

1.4 Why the Name `.venv`?

The name is a convention, not a requirement. One could create `mlflow_env` or `myenvironment`. The name `.venv` is useful because it clearly communicates "project environment" and is recognized naturally by many development workflows.

1.5 Anatomy of a Windows Virtual Environment

A typical Windows environment contains the following structure:

Lib/site-packages is where installed Python packages are primarily stored. **Scripts** contains command-line entry points such as `python.exe`, `pip.exe`, and shell-specific activation scripts. **pyvenv.cfg** records configuration information about the environment and its relationship to the base Python installation.

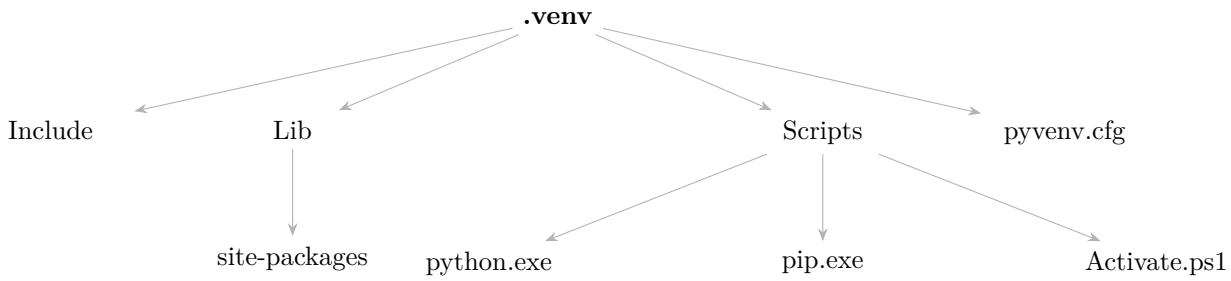


Figure 1.3: Conceptual anatomy of a Windows virtual environment.

Instructor Cue

Ask students to open `.venv` in the VS Code Explorer before activation. This makes the virtual environment tangible: it is not a mysterious "mode" in Python, but an organized directory whose contents can be inspected.

Chapter 2

Activation: What the Shell Actually Changes

2.1 The Command

For Windows PowerShell, the project environment can be activated with:

```
.\.venv\Scripts\Activate.ps1
```

The command is a path to a PowerShell script. Its meaning becomes clear when decomposed:

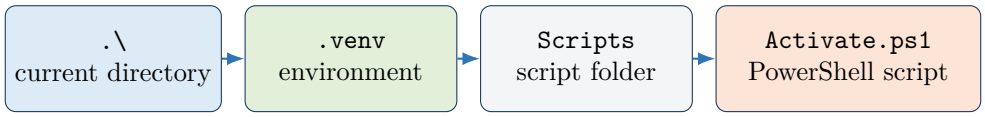


Figure 2.1: Component-wise reading of the PowerShell activation command.

2.2 Component 1: . Means Current Directory

In path notation, `.` denotes the current working directory. If PowerShell is currently at:

```
C:\Users\Student\Desktop\MLFlowHandsOn
```

then `.\.venv` refers to the `.venv` directory inside that location. This is a **relative path**. By contrast, a path beginning with the drive and complete directory chain is an **absolute path**.

Useful demonstrations are:

```
Get-Location
pwd
```

2.3 Component 2: .venv

This identifies the directory containing the project-specific environment. The leading dot is part of the directory name. It is not the same dot as the first `.` that denotes the current directory.

2.4 Component 3: `Scripts`

On Windows, environment executables and activation scripts are conventionally placed in `Scripts`. This includes project-local entry points for Python, pip, and command-line tools installed into the environment.

2.5 Component 4: `Activate.ps1`

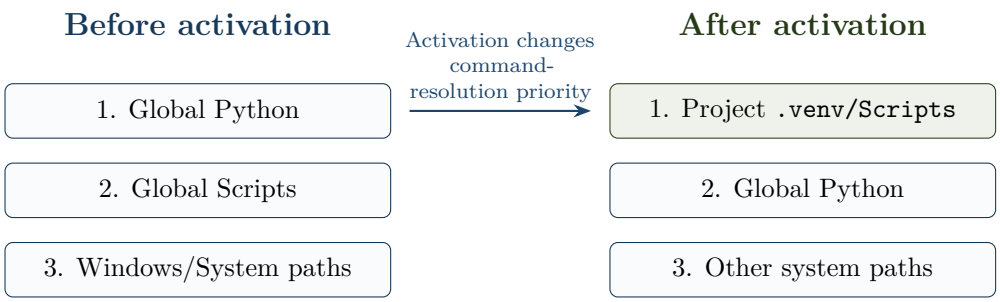
The extension `.ps1` denotes a PowerShell script. Activation is shell-specific, which is why different shells use different files and commands.

Table 2.1: Activation differs because shells and operating systems differ.

Context	Typical activation command
Windows PowerShell	`.\.venv\Scripts\Activate.ps1`
Windows Command Prompt	`.venv\Scripts\activate.bat`
Git Bash on Windows	`source .venv/Scripts/activate`
Linux/macOS Bash	`source .venv/bin/activate`

2.6 Activation Does Not "Start Python"

This distinction is foundational. Activation does not launch a Python process, an MLflow server, a container, or a separate operating system. Instead, it modifies the *current shell environment* so that project-local executables receive priority.



When a student subsequently types `python`, the shell should find `.venv\Scripts\python.exe` before a global interpreter. The same principle applies to `pip` and installed console commands such as `mlflow`.

2.7 Verification Is Better Than Assumption

A mature MLOps habit is to verify the active execution context rather than trusting visual cues alone.

```
where.exe python
Get-Command python
python -c "import sys; print(sys.executable)"
```

The final command is especially instructive: Python itself reports the interpreter executable that is actually running.

Checkpoint

A student has understood activation when they can answer all three questions without memorization: (1) What file did we execute? (2) What shell state changed? (3) Which Python executable will now be selected, and how can we prove it?

2.8 What the Prompt Marker Means

After activation, PowerShell commonly displays `(.venv)` before the prompt. This is a visual cue that the activation script has modified the shell prompt. It is not proof that a Python program is running. Verification should still use interpreter-resolution commands when correctness matters.

Chapter 3

From Virtual Environments to Reproducible ML

3.1 Installing MLflow into the Intended Environment

Once the environment is active, install MLflow with:

```
python -m pip install --upgrade pip
python -m pip install mlflow
mlflow --version
```

Using `python -m pip` is pedagogically valuable because it makes the relationship explicit: the selected Python interpreter is being asked to execute its pip module. This reduces ambiguity when multiple Python installations exist.

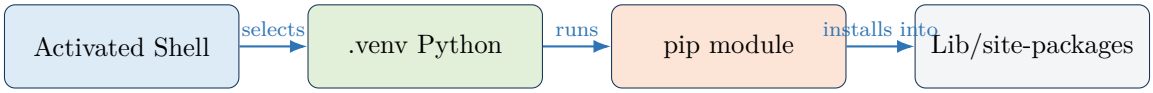


Figure 3.1: The dependency installation chain students should visualize.

3.2 Reproducibility Is Larger Than a Model File

Saving only a trained model artifact does not preserve the full conditions under which the model was produced. Reproducible ML is a composition of several controlled elements.

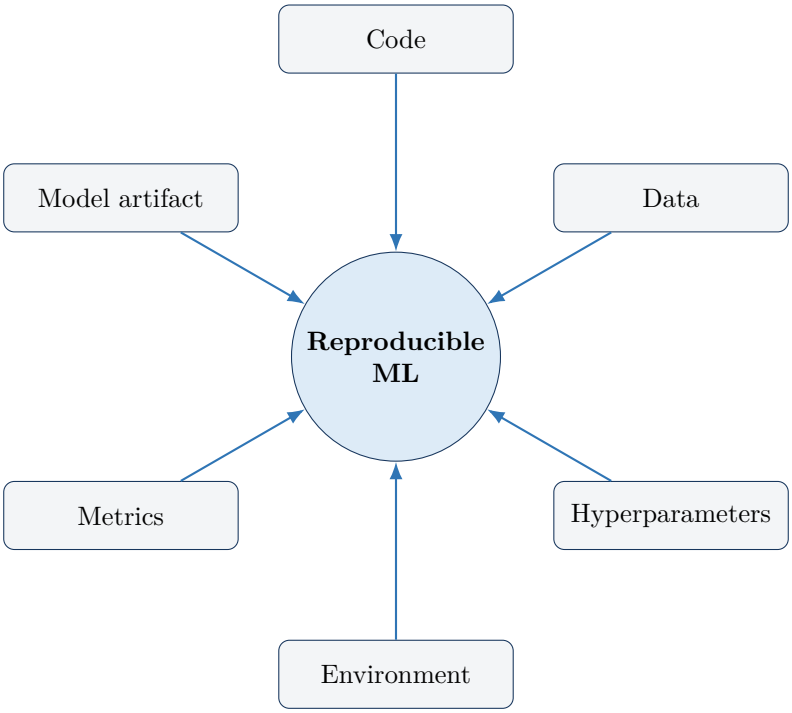


Figure 3.2: A model artifact is only one component of reproducibility.

The **environment** branch includes the Python version and dependency versions. Virtual environments help isolate these dependencies; dependency specifications help record them; MLflow then provides mechanisms for tracking experiments, parameters, metrics, artifacts, and model-related metadata.

MLOps Connection

The conceptual bridge is important: **virtual environments control where software dependencies live; MLflow controls and records the experimental lifecycle around the model.** Students should see these as complementary layers rather than unrelated tools.

3.3 A Layered Mental Model

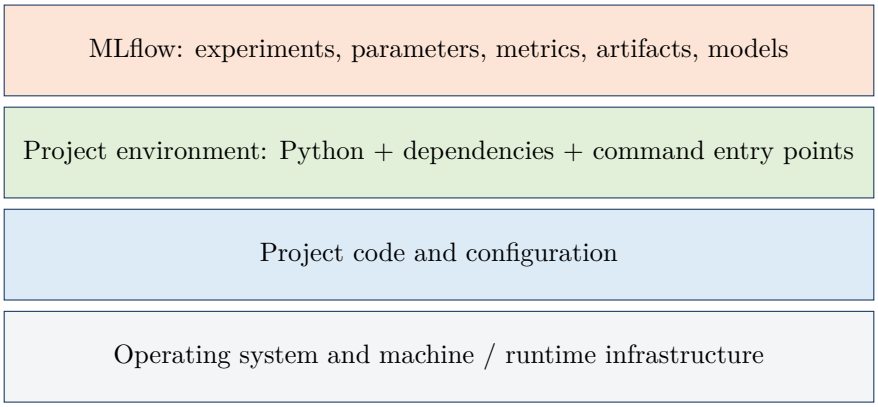


Figure 3.3: A layered view prevents students from treating MLflow as a replacement for environment management.

3.4 Classroom Demonstration: Prove Where Commands Come From

A useful live exercise is to run the following before and after activation:

```
where.exe python
where.exe pip
python -c "import sys; print(sys.executable)"
python -m pip --version
```

Students should compare the resolved paths and explain why they changed. This converts activation from a memorized ritual into an observable system behavior.

Chapter 4

Course Progression: From Local Setup to MLOps Practice

This manual is designed to grow with the course. The following progression provides a conceptual spine for future chapters.

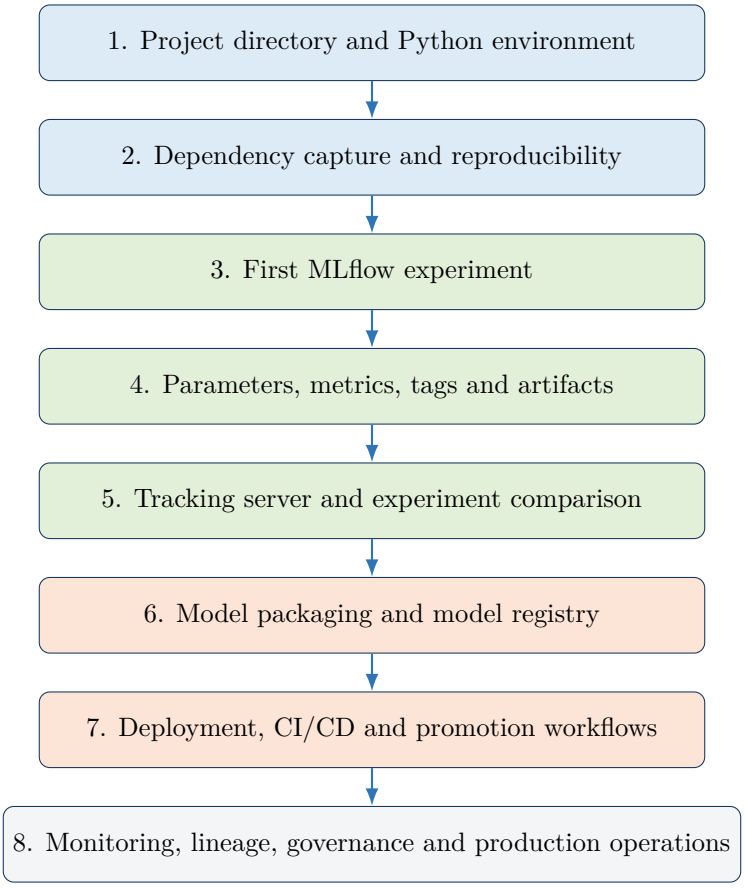


Figure 4.1: Planned growth path for the living manual.

4.1 Recommended First-Lab Sequence

1. Create a dedicated project directory.
2. Open the directory as the VS Code workspace.

3. Create `.venv` with `python -m venv .venv`.
4. Inspect the generated directory before activation.
5. Activate it using the shell-appropriate command.
6. Verify the interpreter path rather than relying only on the prompt marker.
7. Upgrade pip and install MLflow using `python -m pip`.
8. Verify the MLflow installation and record package versions.
9. Only then begin the first tracked ML experiment.

Instructor Cue

At each practical, require students to answer four questions: **What did we execute? What changed? How do we verify it? Why does MLOps care?** Repeating this pattern develops operational reasoning rather than command memorization.

4.2 Planned Future Chapters

The project structure deliberately leaves room for chapters on MLflow Tracking, run anatomy, logging APIs, autologging, artifacts, local and remote tracking servers, backend and artifact stores, model signatures, environments, Model Registry, model serving, CI/CD integration, containerization, cloud deployment, monitoring, lineage, security, governance, and capstone workflows.

Chapter 5

From an Isolated Environment to a Running MLflow Server

The previous chapter established an important foundation: activation selects the project-local Python environment. We can now install MLflow into that controlled environment and start the first local MLflow service.

5.1 Installing MLflow into the Project Environment

With `(.venv)` active, install MLflow using:

```
python -m pip install mlflow
```

Although the shorter command

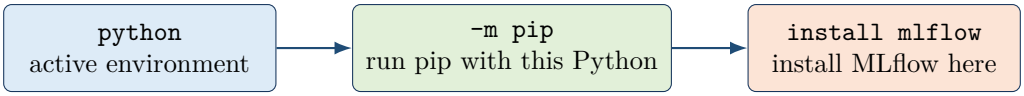
```
pip install mlflow
```

also works when the environment is correctly activated, the form

```
python -m pip install mlflow
```

is particularly useful for teaching because it explicitly connects the package installer to the Python interpreter currently being used.

Conceptually,



The important MLOps principle is not merely that MLflow has been installed. It is that the tool has been installed into a known project execution environment.

Checkpoint

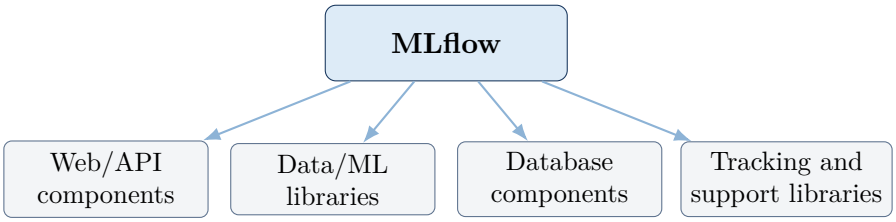
Students should be able to explain why `python -m pip install mlflow` provides a clearer connection between the selected Python interpreter and the package installer than blindly executing a `pip` command.

5.2 One Package Name, Many Dependencies

Installing MLflow causes pip to install MLflow together with a substantial dependency graph.

This should not be interpreted as "many unrelated programs being installed." A modern Python framework is normally constructed from many specialized libraries.

A useful mental model is:



The individual dependency names are less important at this stage than the principle:

> Installing a high-level framework causes the package manager to resolve and install the dependency graph required by that framework.

We will examine individual MLflow components only when they become relevant to an MLOps capability.

5.3 Verify MLflow Before Starting It

Installation and execution are different events.

After installation, verify that the command is available:

```
mlflow --version
```

A stronger environment-level verification is:

```
Get-Command mlflow
```

The second command helps establish whether the MLflow executable being resolved belongs to the intended project environment.

This follows the same engineering principle introduced earlier:

> Do not assume the execution environment is correct merely because a command works. Verify what is actually being executed.

5.4 Starting the Local MLflow Server

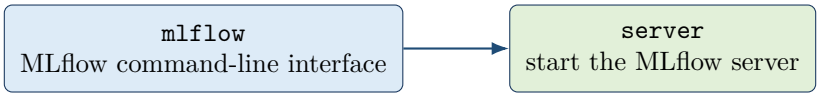
The next important command is:

```
mlflow server
```

This command changes the nature of our interaction with MLflow.

Until this point, MLflow existed only as installed software inside the virtual environment. The `server` subcommand starts a long-running MLflow service.

Component-wise:



5.5 What Happens When the Server Starts?

For a default local setup, several important things happen:

1. MLflow starts a web-accessible service.
2. A backend store is selected for persistent MLflow metadata.
3. If the default database does not yet exist, its tables are initialized.
4. The service begins listening for requests on a network address and port.

In our local execution, MLflow reports:

```
Backend store URI not provided. Using sqlite:///mlflow.db
Registry store URI not provided. Using backend store URI.
...
Uvicorn running on http://127.0.0.1:5000
```

These few lines reveal much more about MLflow architecture than the long package-installation output.

5.6 Understanding `sqlite:///mlflow.db`

The message

```
Backend store URI not provided. Using sqlite:///mlflow.db
```

means that no alternative backend store was explicitly configured, so MLflow selected a local SQLite database named:

```
mlflow.db
```

At this stage, students should understand the backend store simply as the persistent location in which MLflow maintains structured metadata needed by the tracking system.

Examples of such metadata will later include information associated with experiments and runs.

The architecture can initially be viewed as:

This is intentionally a simplified architecture. Artifact storage and more advanced backend configurations will be introduced when the experiments actually begin producing artifacts.

5.7 Understanding `127.0.0.1:5000`

The server reports an address similar to:

```
http://127.0.0.1:5000
```

This address has three conceptually different components:

Table 5.1: Reading the local MLflow server address.

Component	Meaning
`http://`	Communication protocol used by the browser/client to communicate with the local web service.
`127.0.0.1`	The loopback address. It refers to the local machine itself.
`:5000`	The TCP port on which this MLflow service is listening.

Therefore,

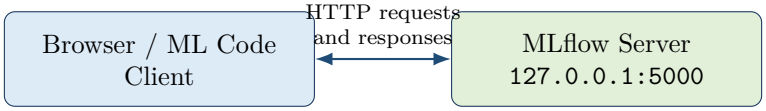
`http://127.0.0.1:5000`

can be read as:

> Use HTTP to communicate with the MLflow service listening on port 5000 of this computer.

5.8 The First Client–Server Mental Model

This is our first important transition from running individual Python commands to interacting with an MLOps service.



The browser and the MLflow server are therefore not the same thing. The browser is a **client**; MLflow is providing a **service** to that client.

5.9 Why the Terminal Remains Occupied

After executing:

```
mlflow server
```

the PowerShell prompt does not immediately return because the server is a long-running process.

The terminal is currently attached to that process and displays its logs.

This is fundamentally different from a command such as:

```
mlflow --version
```

which performs a short operation, prints its result, and terminates.

The distinction is:

Short-lived command	**Long-running service**
starts	starts
↓	↓
performs task	waits for requests
↓	↓
terminates	continues running
↓	↓
prompt returns	terminal shows logs

The local server can normally be stopped from the terminal using:

```
Ctrl+C
```

5.10 Do Not Overinterpret Startup Warnings

During server startup, libraries may emit warnings concerning deprecated interfaces, optional capabilities, or platform-specific features.

A warning is not automatically a server failure.

The important distinction is between:

Warning: execution can often continue, although some condition deserves attention.

Error: an operation has failed and may prevent the requested functionality from completing.

Successful startup: the server reports that it is listening and application startup has completed.

For this first lab, the decisive observation is that the MLflow server successfully reaches its listening state.

Checkpoint

Before moving to experiment tracking, students should be able to explain:

1. where MLflow was installed;
2. what `mlflow server` starts;
3. why the terminal remains occupied;
4. what `127.0.0.1` represents;
5. what port `5000` represents;
6. why `mlflow.db` appeared; and
7. the difference between the MLflow client, server, and backend store.